

RulePI: Textual Policy Iteration for Interactive Agents

Anonymous ACL submission

Abstract

An interactive agent can use a failure diagnosis to retry a task, but the lesson is discarded and the extra attempt increases deployment cost. We study a narrow alternative: convert recurring diagnostics into persistent textual rules that a frozen actor can reuse. RulePI performs one policy-iteration step over text: collect training trajectories, diagnose failure modes, propose reusable rules, and optionally retain an update only when held-out return improves. Across ten independently generated TextWorld-24 suites (240 paired games), persistent rules improve success from 33.8% to 56.7% (paired 95% CI: +17.1 to +29.2 points). Across ten supported TextArena environments (130 paired episodes), success improves from 19.2% to 61.5% (CI: +18.0 to +72.0 points). Persistent rules attain similar observed success to task-local diagnostic retry while using about half as many test actions. Two negative results define the boundary: validation gating does not improve success over ungated persistence, and a Qwen2.5-7B actor remains at 25.0% success. Thus, the evidence supports reusable textual rules for structured actors, not generic language-model self-improvement.

1 Introduction

Interactive agents often receive useful feedback only after they fail. A common response is to append a diagnosis and retry the same task, as in verbal reinforcement and reflection methods (Shinn et al., 2023). This can repair a local failure, but it spends another episode and does not answer whether the lesson transfers to later tasks. At the other extreme, fine-tuning an actor may be expensive or unavailable.

We examine the middle ground in which the actor is frozen and the policy is editable text. TextGrad showed that natural-language feedback can optimize prompts and compound AI systems (Yuksekgonul et al., 2025). Our question is de-

liberately smaller and operational: *when the same failure mode recurs, can a persistent textual rule replace repeated task-local diagnosis?*

RulePI treats a collection of textual rules as a policy. It gathers trajectories, maps failures to candidate rules, and evaluates an updated rule set. The deployed policy is shared across test tasks and receives only one attempt per task. We compare it with (i) a fixed policy, (ii) a strong retry baseline that sees a task-local diagnosis after failure, and (iii) ungated persistence, which stores training-derived rules without validation.

Our contribution is a controlled mechanism study with two findings:

- Persistent rules improve success over a fixed policy on TextWorld and TextArena, with paired intervals above zero.
- Persistence attains similar observed success to diagnostic retry with roughly half the test interaction, but validation gating adds no success benefit and does not improve a Qwen2.5-7B actor.

The second finding is important: the supported claim concerns rule reuse by structured actors, not reinforcement learning of model weights or general LLM self-improvement.

2 Related Work

ReAct interleaves language-model reasoning and actions (Yao et al., 2023), while Reflexion stores verbal feedback to improve subsequent attempts (Shinn et al., 2023). ExpeL extracts natural-language insights from training experiences and recalls them on later tasks (Zhao et al., 2023). ProTeGi uses textual critiques, prompt edits, and search to optimize instructions (Pryzant et al., 2023); TextGrad generalizes this idea to prompts and compound systems (Yuksekgonul et al., 2025).

RulePI is closest to ExpeL, but studies a narrower comparison: a reusable text policy versus a task-local diagnosed retry, with paired evaluation and both test-time and optimization interaction counted.

3 RulePI

Let x be a textual policy and let $\pi(a \mid o, x)$ be a frozen actor that selects action a from observation o . RulePI changes x , not the actor parameters. One iteration has four steps.

1. Collect. Run $\pi(\cdot \mid \cdot, x)$ on training tasks and record trajectories $\tau = (o_t, a_t, r_t)_{t=1}^T$, including success, invalid actions, repetition, and episode length.

2. Diagnose. Map failed trajectories to a reusable failure mode, such as `graph_exploration` or `objective_sequence`. The current prototype retrieves a human-authored candidate rule for that mode. For example, a repeated room loop yields: “maintain a room graph, prefer unvisited exits, and avoid immediate backtracking.”

3. Persist. Append the candidate rules to form x' . Unlike diagnostic retry, x' is shared by all later tasks rather than discarded after one episode.

4. Gate (optional). Evaluate old and new policies on disjoint validation games. RulePI accepts x' when

$$\hat{J}_{\text{val}}(x') \geq \hat{J}_{\text{val}}(x) + \delta, \quad (1)$$

where $\hat{J}$ combines task return, success, validity, and efficiency, and $\delta = 0.001$. TextArena additionally requires the paired-bootstrap lower bound on improvement to be nonnegative. The ungated ablation always keeps the training-derived update.

What is being optimized, and where is the RL?

The positive experiments use prompt-aware structured actors. They recognize rule content and activate existing routines for objective sequencing, graph exploration, recipe execution, or game strategy. This makes the experiment easy to inspect: the outer loop selects and reuses a capability; it does not invent that capability or train neural weights. The environment supplies trajectories and scalar return, and the validation comparison is a discrete policy-improvement step. RulePI is therefore textual policy selection under reward, not a policy-gradient or PPO algorithm.

4 Experimental Setup

TextWorld-24. TextWorld generates controlled text games with varied maps and quests (Côté et al., 2018). Each repetition contains six games from each of four families: simple objectives, coin collection, treasure hunting, and cooking. For every repetition, 8 training games, 8 validation games, and 24 test games use disjoint seeds. We run 10 independent repetitions, yielding 240 paired test games per method, with at most 80 actions per attempt.

TextArena-10. We use ten environments for which the actor implements the corresponding rule interface: Nim, Connect Four, Reverse Tic-Tac-Toe, Guess the Number, Frozen Lake, Tower of Hanoi, Lights Out, Mastermind, Blackjack, and Bandit (Guertler et al., 2025). Three seeds per environment are used for training and validation; ten fresh test seeds and both player sides where applicable yield 130 paired test episodes per method.

Baselines. FIXED uses the initial policy once. RETRY+DIAG first uses the fixed policy; after failure it receives the task’s diagnosed rule and retries once, so its maximum test budget is 160 actions in TextWorld. UNGATED learns persistent rules from training trajectories and retains them without validation. RULEPI applies the validation gate in Section 3 and then freezes the policy for test.

Statistics. Methods are paired on identical games, seeds, and target sides. We report hierarchical paired-bootstrap 95% intervals from 10,000 resamples, clustering by TextWorld generation or TextArena environment and resampling episodes within clusters. Optimization actions include environment steps used to produce and evaluate the policy; test actions include both attempts for RETRY+DIAG.

5 Results

5.1 Persistent Rules Improve Success

Table 1 supports the first claim. On TextWorld, RulePI improves success by 22.9 percentage points over the fixed policy; the paired interval excludes zero. On TextArena, the gain is 42.3 points and again excludes zero. Diagnostic retry and persistent rules are close in success: RulePI is 1.25 points higher on TextWorld (CI: 0.0 to 3.3) and tied on TextArena.

Benchmark	Fixed	Retry+Diag	Ungated	RulePI	RulePI – Fixed (95% CI)
TextWorld-24 (240 games)	33.8	55.4	56.7	56.7	+22.9 [17.1, 29.2]
TextArena-10 (130 episodes)	19.2	61.5	61.5	61.5	+42.3 [18.0, 72.0]

Table 1: Success rates (%) for matched test episodes. Intervals are hierarchical paired-bootstrap intervals for RulePI minus the fixed policy.

Family (60 games each)	Fixed	Retry	RulePI
Coin collection	100.0	100.0	100.0
Cooking	0.0	16.7	16.7
Simple objective	0.0	11.7	11.7
Treasure hunting	35.0	93.3	98.3

Table 2: TextWorld success (%) by game family.

Benchmark	Test actions			N^*
	Retry	RulePI	Opt. actions	
TextWorld	81.78	42.25	7,965	202
TextArena	20.68	10.19	7,921	756

Table 3: Mean actions per deployed task, one-time RulePI optimization actions, and interaction break-even count N^* .

Where do the TextWorld gains occur? Table 2 shows that RulePI improves three families and leaves the already-solved coin family unchanged. The largest gain is on treasure hunting, where the exploration rule directly addresses repeated room loops. Cooking and simple objectives remain difficult, which prevents the overall result from being read as universal task solving.

5.2 Persistence Reduces Deployment Interaction

Table 3 supports the second claim. RulePI uses one test attempt and about half as many test actions as diagnostic retry: 42.25 versus 81.78 on TextWorld and 10.19 versus 20.68 on TextArena. The paired reductions are 39.53 actions (CI: 34.17 to 44.95) and 10.48 actions (CI: 7.35 to 14.19), respectively.

This is an amortization claim, not a free-compute claim. If C_{opt} is optimization interaction and $A_r - A_p$ is the per-task saving over retry, the break-even deployment count is $N^* = C_{\text{opt}} / (A_r - A_p)$. RulePI crosses this point within the 240-game TextWorld study, but not within the 130-episode TextArena study.

5.3 A Matched Example

Table 4 traces one logged test game, `treasure_level_15`. The fixed policy

Method	Attempts	Actions	Outcome
Fixed	1	80	loop; failure
Retry+Diag	2	86	success
RulePI	1	6	success

Table 4: Matched test example. Retry+Diag uses 80 failed actions followed by the same six-action route as RulePI.

repeats the same four-direction loop until the 80-action horizon. Both improved methods then execute the same six-action route, but Retry+Diag first pays for the failed attempt. RulePI learned the reusable exploration rule only from disjoint training games and applies it on its first test attempt.

5.4 What the Evidence Does Not Show

The gate does not improve success. Gated and ungated persistence are identical on TextWorld success, reward, and test actions. On TextArena they tie in success and test actions; gating retains 6 rather than 10 rules and changes mean reward only slightly (0.573 versus 0.564). Thus, persistence explains the observed success gain; validation is only a conservative diagnostic in these runs.

The LLM boundary is negative. We also run Qwen2.5-7B at temperature 0.7 on 24 TextWorld games with an 80-action horizon. The fixed and RulePI protocols both obtain 25.0% success. The candidate is rejected because validation score decreases from 0.508 to 0.506, so the deployed RulePI policy remains the base policy. The structured-actor result therefore does not establish generic prompt optimization for small LLM agents.

6 Practical Guidance

RulePI is useful when (i) many future tasks share failure modes, (ii) the actor reliably executes written rules, and (iii) the policy will be reused enough to amortize evaluation. For a small number of tasks, diagnostic retry is simpler and can cost less overall. A practical deployment should first mea-

sure the action saving $A_r - A_p$, estimate N^* , and retain a gate only if rejected updates or validation regressions occur in practice.

7 Conclusion

Persistent textual rules improve two controlled interactive-agent suites and attain comparable observed success to task-local diagnostic retry with substantially less deployment interaction. The clean result is rule reuse, not a benefit from validation gating and not LLM self-improvement. This narrower conclusion makes RulePI an interpretable testbed for studying when trajectory lessons should become shared policies.

Limitations

The positive experiments use structured actors that detect human-authored rule content and dispatch pre-existing routines. Candidate rules are selected from a predefined library keyed by benchmark failure modes; they are not freely generated from raw trajectories. Consequently, the study measures rule selection and reuse, not the acquisition of new skills or general natural-language instruction following. The task-local retry baseline receives the same diagnosed rule after its first failed test attempt, while persistent methods learn only from training data; this is intentionally a strong deployment-time baseline but uses a larger test budget. The Qwen2.5-7B boundary has only 24 games and no multi-seed confidence interval. TextWorld rejected some requested generation seeds; the runner deterministically advanced to the next valid seed and logged requested and actual seeds before evaluation. Finally, interaction counts measure environment actions, not wall-clock time, energy, or textual-optimizer inference cost.

Ethical Considerations

The experiments use synthetic games and no personal or sensitive data. Persistent agent rules can nevertheless amplify misspecified rewards or unsafe behavior; deployments should log rule provenance, validation decisions, and regressions, and should restrict which rules can be activated in consequential environments.

References

- Marc-Alexandre Côté, Ákos Kádár, Xingdi Yuan, Ben Kybartas, Tavian Barnes, Emery Fine, James Moore, Ruo Yu Tao, Matthew Hausknecht, Layla El Asri, Mahmoud Adada, Wendy Tay, and Adam Trischler. 2018. [TextWorld: A learning environment for text-based games](#). *Computing Research Repository*, arXiv:1806.11532.
- Leon Guertler, Bobby Cheng, Simon Yu, Bo Liu, Leshem Choshen, and Cheston Tan. 2025. [TextArena](#). *Computing Research Repository*, arXiv:2504.11442.
- Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. [Automatic prompt optimization with “gradient descent” and beam search](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7957–7968. Association for Computational Linguistics.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. [Reflexion: Language agents with verbal reinforcement learning](#). In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. [ReAct: Synergizing reasoning and acting in language models](#). In *International Conference on Learning Representations*.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. [Optimizing generative AI by backpropagating language model feedback](#). *Nature*, 639:609–616.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2023. [ExpeL: LLM agents are experiential learners](#). *Computing Research Repository*, arXiv:2308.10144.

A Reproducibility Details

All four methods are evaluated on identical test instances. For TextWorld, requested outer seeds are spaced by 100,000; generator-invalid requests are advanced by one and recorded in a manifest. Within each repetition, training and validation seeds are offset by 10,000 and 20,000 from the test seed. The textual optimizer adds at most four rules in TextWorld and evaluates candidate policies under the same 80-action horizon used at test. The three reusable TextWorld rules cover ordered objective execution, graph exploration, and cookbook-based recipe completion.